ODT

Week 9

```python
from machine import Pin

button = Pin(5,Pin.IN,Pin.PULL_UP)

val_1 = button.value()
```

```python
from machine import Pin

button = Pin(5,Pin.IN,Pin.PULL_UP)

while True :
    val_1 = button.value()

    if val_1 == 0:
        print("Monday")
        time.sleep(1)

    if val_1 == 0:
        print("Tuesday")
        time.sleep(1)

    if val_1 == 0:
        print("Wednesday")
        time.sleep(1)

    time.sleep(0.1)
```

```python
from machine import Pin

button = Pin(5,Pin.IN,Pin.PULL_UP)

while True :

    if button.value() == 0:
        print("Monday")
        time.sleep(1)

    if button.value() == 0:
        print("Tuesday")
        time.sleep(1)

    if button.value() == 0:
        print("Wednesday")
        time.sleep(1)

    time.sleep(0.1)
```

```python
from machine import Pin

button = Pin(5,Pin.IN,Pin.PULL_UP)

while True :
    val_1 = button.value()

    if val_1 == 0:
        print("Monday")
        time.sleep(1)

    if val_1 == 0:
        print("Tuesday")
        time.sleep(1)

    if val_1 == 0:
        print("Wednesday")
        time.sleep(1)

    time.sleep(0.1)
```

- button.value() is **called only once** in the loop.
- That value is stored in the variable val_1.
- All three if statements **use the same stored value**.

During one loop iteration:

Button is checked → 1 time

Even if the button state changes during execution, the program **will not notice**, because it keeps using the same stored value.

```python
from machine import Pin

button = Pin(5,Pin.IN,Pin.PULL_UP)

while True :

    if button.value() == 0:
        print("Monday")
        time.sleep(1)

    if button.value() == 0:
        print("Tuesday")
        time.sleep(1)

    if button.value() == 0:
        print("Wednesday")
        time.sleep(1)

    time.sleep(0.1)
```

- button.value() is called **inside every if statement**.
- Each if **reads the button again from hardware**.

During one loop iteration:

Button is checked → 3 times

This means the program keeps re-reading the button state.

```
button.value()
```

Reads the button state from the hardware

Time Measurement

ESP32 AND MICROPYTHON

TIME MEASUREMENT

- Import the time module
- Create any variable to store the absolute time using time.time()
- Print the variable

TIME MEASUREMENT

- `time.time()` returns the seconds elapsed since 1st Jan 2000
- Absolute time
- **Fixed Reference : 1st Jan 2000**
- Unit : Seconds

TIME MEASUREMENT

- `time.ticks_ms()` returns the number of milliseconds elapsed since the microcontroller started running.
- Relative time
- Units : Milliseconds

TIME MEASUREMENT

- `time.ticks_us()` returns the number of microseconds elapsed since the microcontroller started running.
- Relative time
- Units : Microseconds

TIME MEASUREMENT

- Time taken by a particular task??

Time variable t1 – Store time

Task

Time variable t2 – Store time

Difference between t2 & t1

TIME MEASUREMENT

- Measure code execution time for printing your name 5 times **without** for loop

- Measure code execution time for printing your name 5 times **with** for loop

TIME MEASUREMENT

- time.ticks_ms() resets after **≈ 12.4 days**
- time.ticks_us() resets after **≈ 17.9 minutes**

- time.ticks_diff(end, start)

STOPWATCH

USING TWO PUSH BUTTONS

- When START button is pressed:
  - Record the **start time** in a variable using time.ticks_ms()

- When STOP button is pressed:
  - Record the **end time** in a variable
  - Calculate the **elapsed time using time.ticks_diff(end,start)**
  - Print the elapsed time

Make sure you put a delay of 0.1s for system stability

TIME PERCEPTION : 5 SECONDS

HOW ACCURATELY YOU CAN ESTIMATE 5 SECONDS USING YOUR INTERNAL SENSE OF TIME AND COMPARE IT WITH THE **ACTUAL TIME MEASURED BY THE ESP32**.

- When START button is pressed:
  - Print **"Start estimating 5 seconds!" & LED ON indicating start**
  - Record the **start time** in a variable using time.ticks_ms()

- When STOP button is pressed:
  - Record the **end time** in a variable **& LED OFF indicating stop**
  - Calculate the **elapsed time using time.ticks_diff(end,start)**
  - Print how close the time was to **5 seconds(5000 milliseconds)**

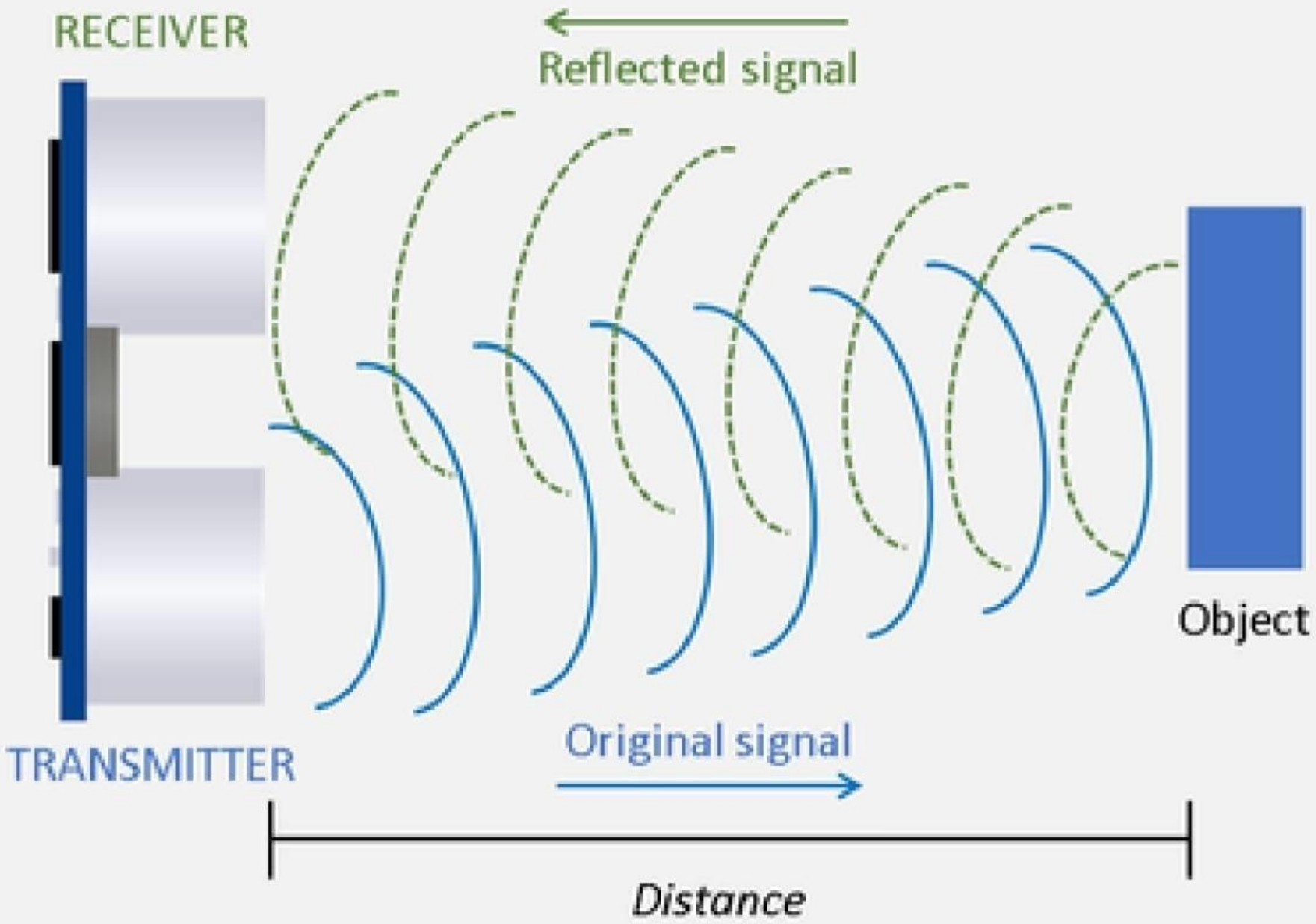
Make sure you put a delay of 0.1s for system stability

REACTION TIME

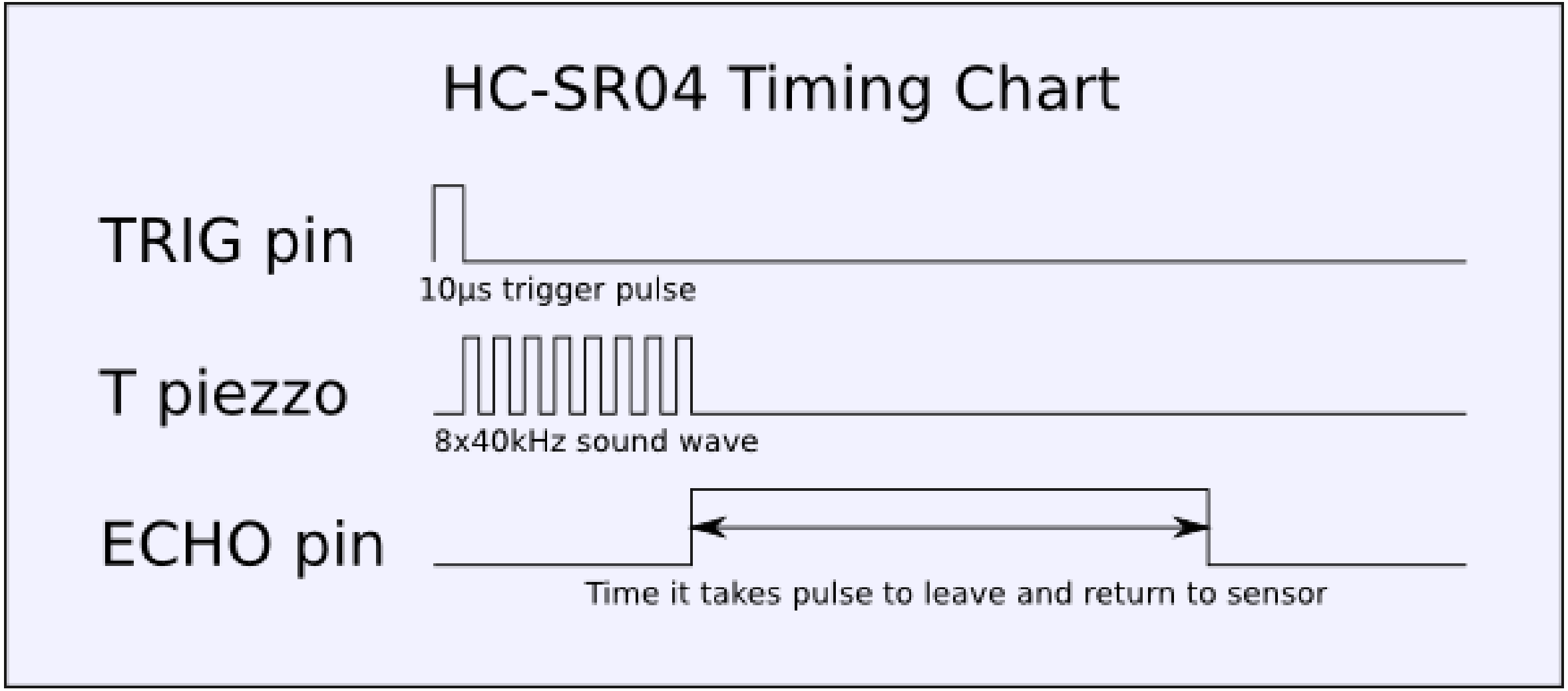
Game Flow:

1. **Idle State:**
   LED blinks rapidly to indicate the system is ready.
2. **Start Game:**
   When **Button** is pressed:
   - LED in ON state for 2 seconds to signal the game is starting.
   - LED turns OFF.
3. **Reaction Test:**
   - After a **random delay (1–5 seconds)**, the **LED turns ON**.
   - Press **Button as quickly as possible** when the LED turns ON.
4. **Result:**
   - The program measures and prints your **reaction time in milliseconds**.

HC-SR04



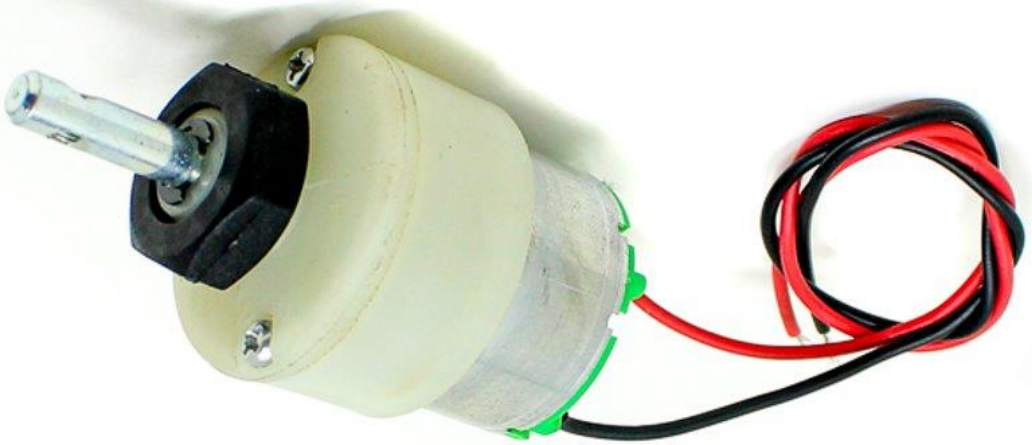
Ultrasonic Sensor


HC-SR04 Timing Chart
TRIG pin
10µs trigger pulse
T piezzo
8x40kHz sound wave
ECHO pin
Time it takes pulse to leave and return to sensor

DC MOTOR

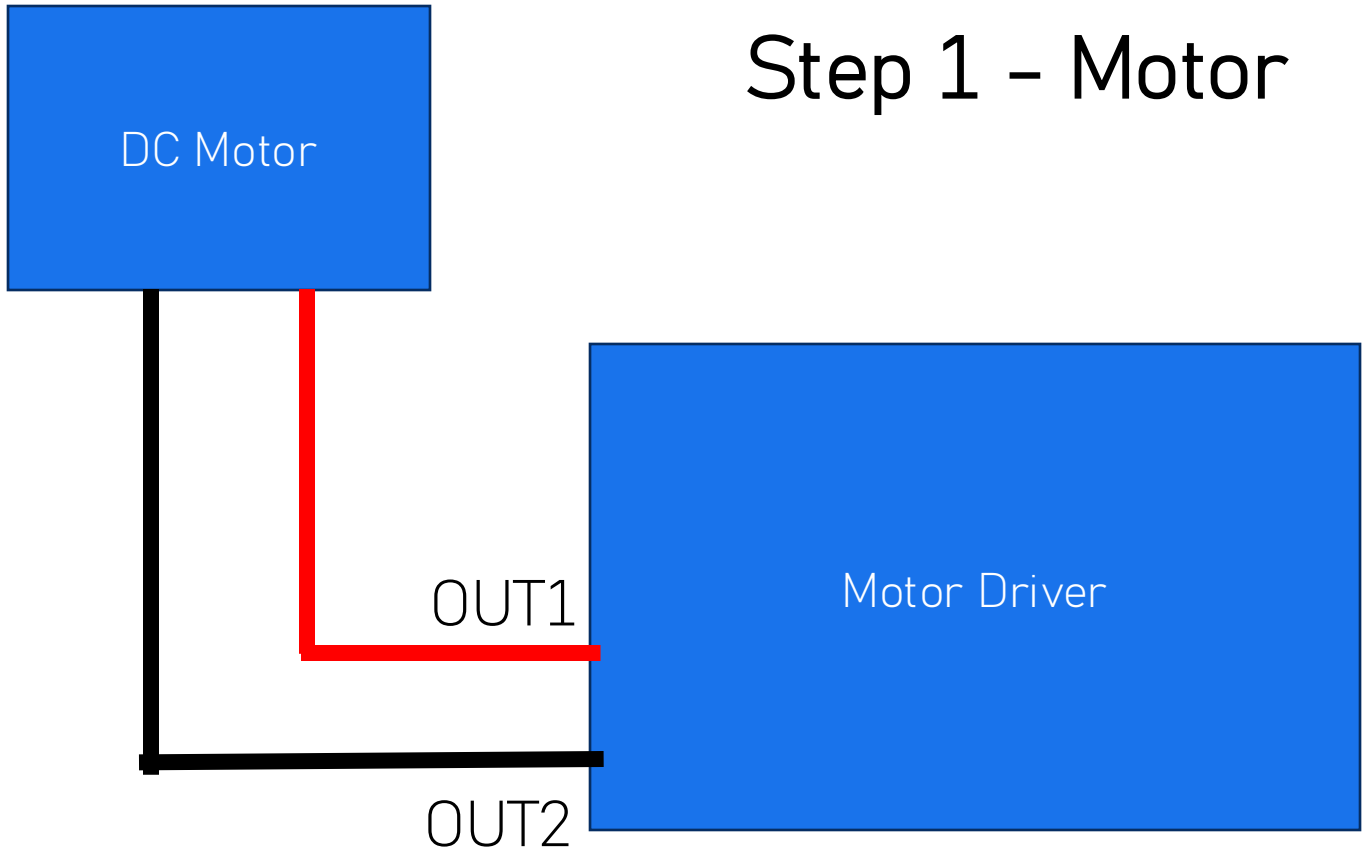
Power Adaptor

DC Motor

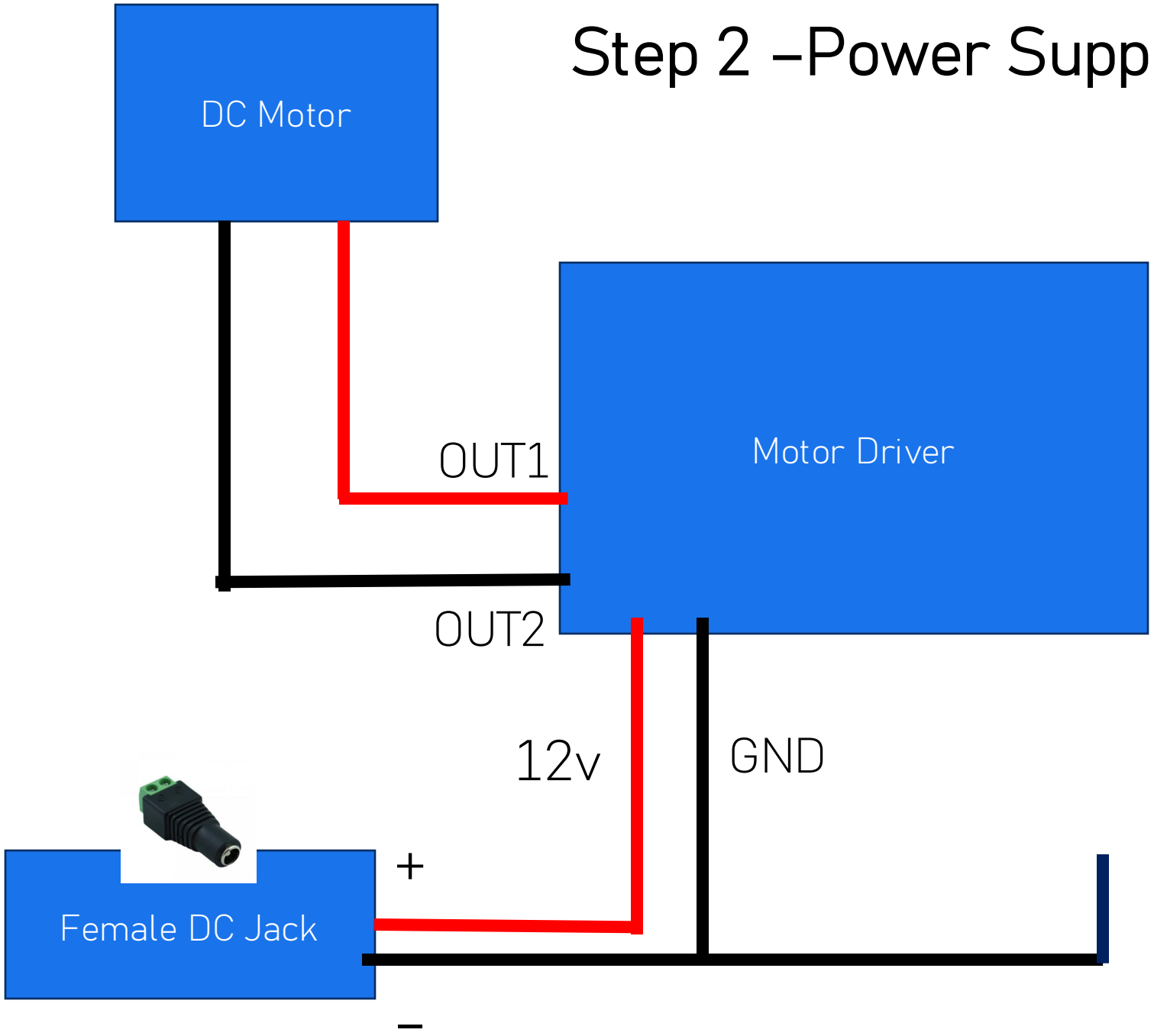


DC Female Jack Connector

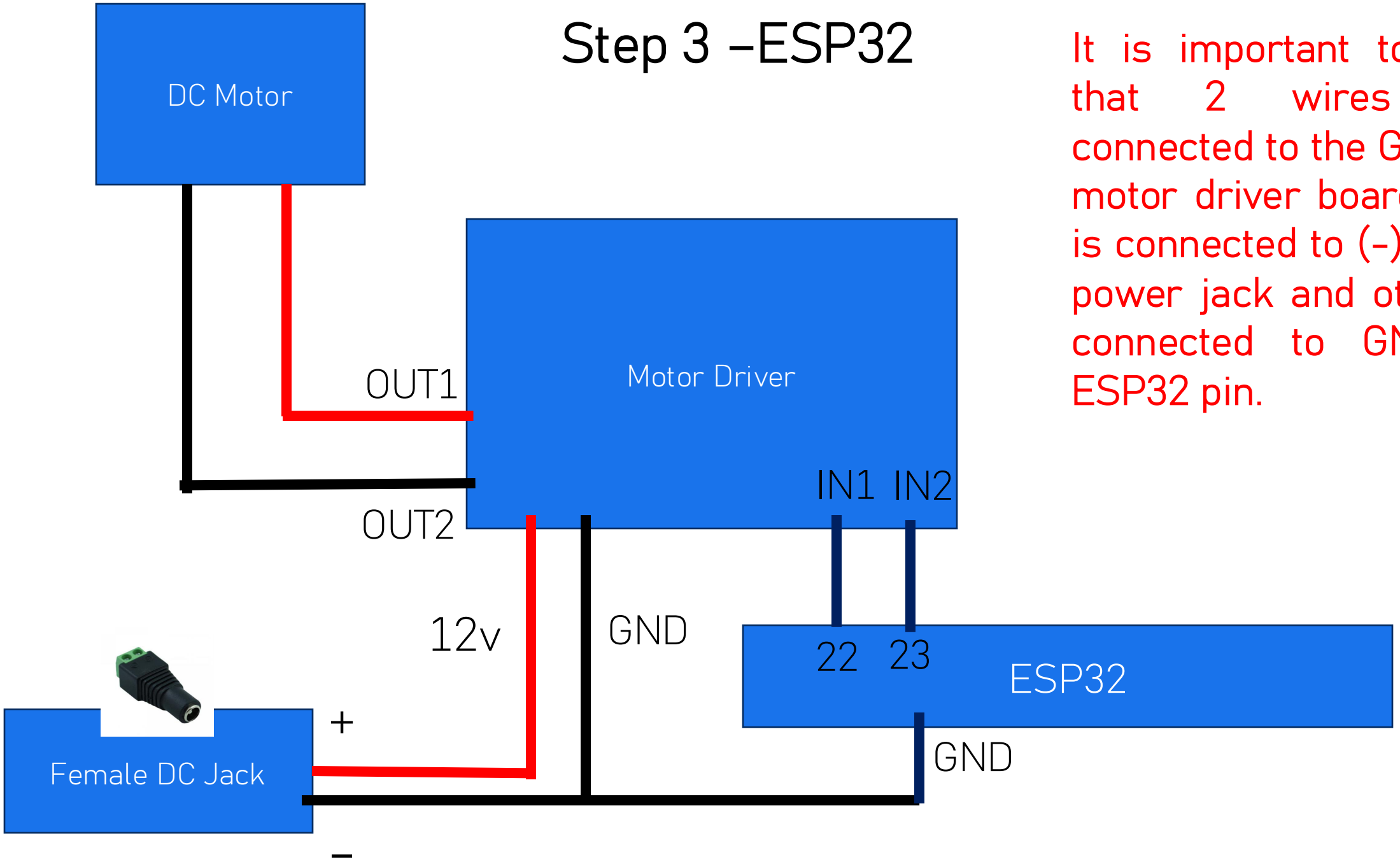
Step 1 - Motor



Step 2 –Power Supply

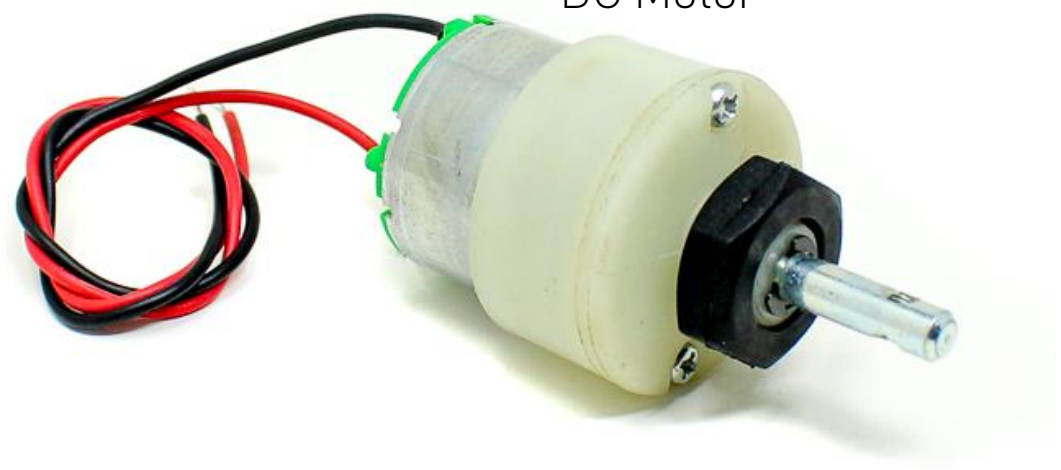


Step 3 –ESP32



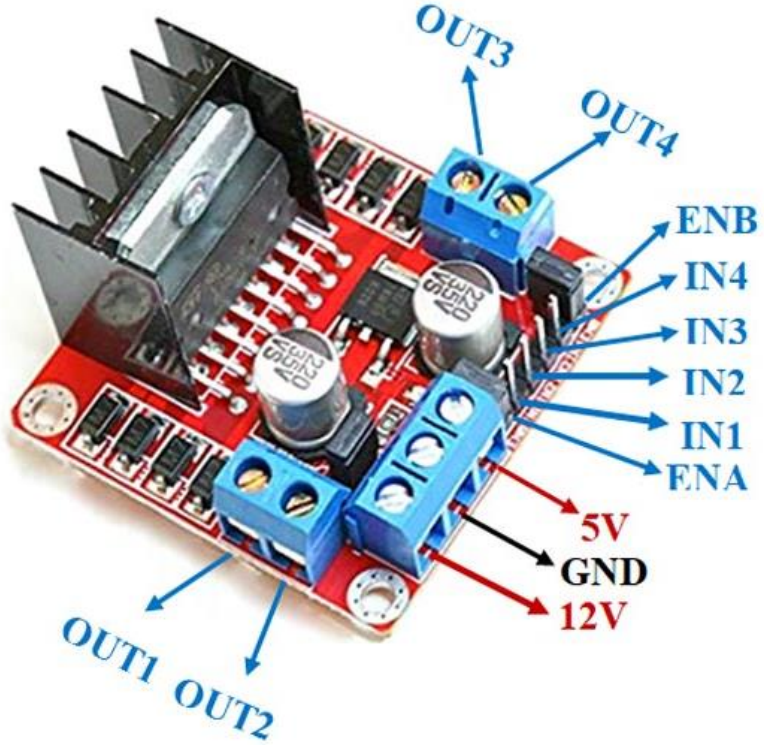
It is important to note that 2 wires are connected to the GND on motor driver board. One is connected to (-) of DC power jack and other is connected to GND of ESP32 pin.

DC Motor



Power Adaptor

Motor Driver



DC Power Jack

IN1	D12
IN2	D13
ENA	D15
OUT1	Motor Red
OUT2	Motor Black
GND	- DC Power Adaptor & ESP32
12v	+ DC Power Adaptor

Interfacing

Coding : Direction Control

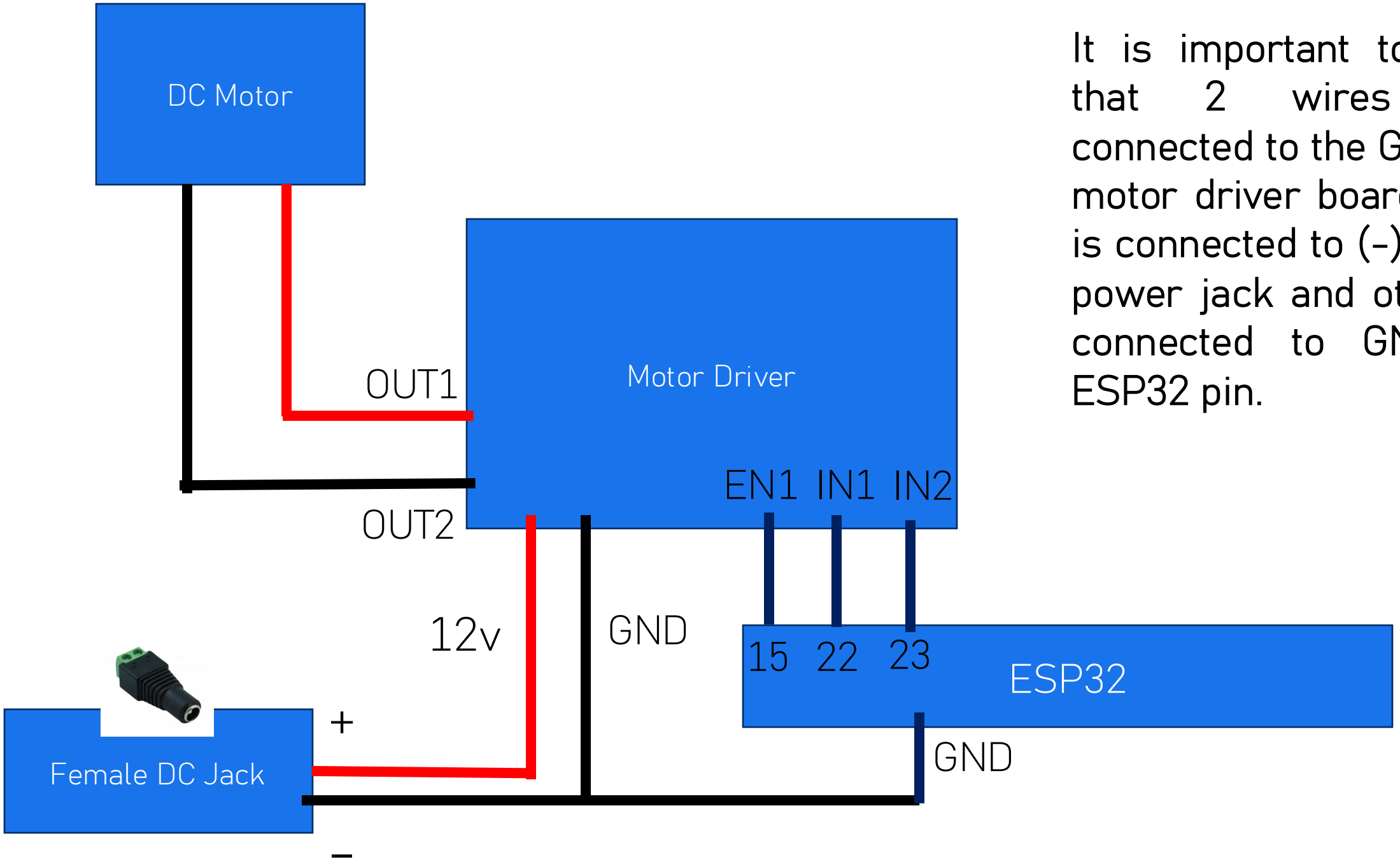
```
#import library for Pin initialization

#assign IN1 and IN2 as connected to GPIOs

#initiate a loop to run forever

    # IN1 pin to ON
    # IN2 pin to OFF
    # Delay 3 seconds
    # IN1 pin to OFF
    # IN2 pin to ON
    # Delay 3 seconds
```

Once you run the code, you should be able to see the rotation.



It is important to note that 2 wires are connected to the GND on motor driver board. One is connected to (-) of DC power jack and other is connected to GND of ESP32 pin.

Coding : Direction & Speed Control

```
#import library for Pin initialization and PWM

#assign IN1 and IN2 as connected to GPIOs

#assign ENA as PWM pin

#set the duty-cycle and frequency of the PWM pin to roughly 75% and 1000 Hz respectively
#Duty cycle corresponds to speed/rpm (Range : 60%(Min Speed) - 100%(Max Speed))
#initiate a loop to run forever

    #set IN1 pin ON
    #set IN2 pin OFF
    #Delay 3 seconds
    #set IN1 pin ON
    #set IN2 pin OFF
    #Delay 3 seconds
```

Learning Objectives : Week 9

- Time Measurement using time.time(), time.ticks_ms() and time.ticks_us()
- Distance Measurement Using HC-SR04 Ultrasonic Sensor
- DC Motor